The Architecture of Intent: A Framework for Designing Delegated Systems

Marcel Aldecoa *Independent practitioner*

Paper status: Workshop-length variant (paper v0.1-workshop) compressed from the long-form arXiv version. 10–12 pages target. Position-and-framework paper, same load-bearing commitments as the long form; intended for workshop or short-form journal submission. The long form is the citation reference; this version is the gateway.

Framework version: v2.4.0 (2026-05-10). Both this paper and its companion book reflect the same framework version. See `CHANGELOG.md` at the repository root for the versioning convention and release history.

Target venue: workshop or short-form journal; the long form remains the arXiv reference.

Abstract

Delegated systems — software, organizations, automated pipelines, and increasingly AI agent systems — share a structural problem: humans must express intent precisely enough that a non-human executor can act on it without supervisory rescue. We present *The Architecture of Intent*, a framework that treats intent as a primary design artifact distinct from implementation, operationalized through four load-bearing elements. **Archetypes** commit a system to one of five canonical delegation shapes (Advisor, Executor, Guardian, Synthesizer, Orchestrator) before design begins. **Four orthogonal calibration dimensions** — agency, autonomy, responsibility, reversibility — refactor Shavit & Agarwal's (Shavit, Agarwal, et al. 2023) operational variables into independent levers, decomposing what SAE J3016 (SAE International 2021) packs into a single "automation level." A **fix-locus failure taxonomy** (Cat 1–7) partitions failures by which artifact must change, complementing Cemri et al.'s (Cemri et al. 2025) empirical symptom-locus taxonomy (MAST). **Spec-Driven Development** (GitHub 2024) is the executable protocol that binds the four elements into a specification an agent can run and a human can validate. Three contributions are explicitly novel: the orthogonality operationalization of agency and autonomy, the fix-locus framing of the failure taxonomy, and **Cat 7 (Perceptual Failure)** for perceiving-then-acting systems (computer-use agents, browser-use agents). We position the work as a position-and-framework paper without empirical validation at scale and argue the framework's reach extends beyond AI agent systems to delegated systems generally. The companion book (Aldecoa 2026) walks the discipline end-to-end across three running scenarios.

Keywords: intent engineering, agentic development lifecycle, spec-driven development, agent governance, AI safety, software architecture.

---

1. Introduction

Software has always been a delegation, but the *width* of delegation in 2024–2026 changes structural facts that prior software-engineering literature could leave implicit. A single human action — writing a brief specification, opening a ticket, asking a question — is now enough to commit a system to a long sequence of consequential decisions made by an executor that is not human and

that, even when equipped with explicit clarification APIs, exhibits a documented *execution-first bias*: defaulting to confident continuation rather than to interactive escalation when the spec is ambiguous.

This is not a capability claim. By 2026, frontier coding agents ship with first-class clarification mechanisms — Anthropic's Claude Code exposes an `AskUserQuestion` tool (Anthropic 2024--2025); GitHub Copilot, Cursor, and Cline support interactive sessions. What is unreliable is the *judgment* about when to invoke them. The structural cause is the post-training reward shape: RLHF and related preference-optimization regimes reward task *completion* over task *clarification*. Pausing to ask is a low-frequency behavior in the reward signal; one-shotting is the high-probability default. The agent has the tool; the agent does not reliably use it.

Two observations frame the gap the framework addresses. **First**, human professionals tolerated imprecise intent because they exercised silent judgment to bridge it: senior engineers supplied missing constraints from experience, escalated ambiguity rather than executing it, and rewrote underspecified tickets into well-scoped commits without updating the specification document. None of that judgment was visible to the spec, which retained its underspecified form because the implementer absorbed the gap. None of it transfers to an automated executor that interprets the document literally. **Second**, automated executors make imprecise intent immediately visible as wrong outputs: where a senior engineer would have surfaced an ambiguity, an LLM agent typically resolves it probabilistically against the training distribution and proceeds. The cost per call is small — a slightly off-target output, a near-miss at the constraint boundary — but compounds across a deployment, and post-incident reconstruction is hard precisely because no single call was visibly wrong.

The framework's central claim follows: the gap between intent and implementation has not grown; it has merely become *visible*. Spec authors in 2026 are paying explicit costs for what spec authors in 2006 paid implicit costs for. This paper proposes a framework that makes intent thick enough to be governable when implementation is automated.

Contribution. The paper develops a framework with four load-bearing elements (§3) and instantiates it against AI agent systems (§4). Three contributions are *novel*, narrowly framed: (i) the operationalization of autonomy and agency as orthogonal axes; (ii) the fix-locus framing of the failure taxonomy; and (iii) **Cat 7 (Perceptual Failure)** as a new category for perceiving-then-acting systems with four sub-categories and structural fixes for each. The paper does *not* claim novelty for SDD as a discipline (lineage from spec-kit and DevSquad), archetypes-as-concept (lineage from Anthropic's *Building Effective Agents* (Anthropic 2024a)), the four dimensions individually (lineage from SAE J3016 and Shavit & Agarwal), or Cat 1–6 as categories (synthesis from common practice). The synthesis is the larger contribution. The paper is a *position-and-framework* paper without empirical validation at scale; §6 names this and the other limitations explicitly.

---

2. Prior work and lineage

The framework operates within four broad bodies of standing literature. **Spec-Driven Development** is direct lineage from GitHub's spec-kit (GitHub 2024) and Microsoft's DevSquad Copilot (Microsoft 2026), with antecedents in Meyer's Design by Contract (Meyer 1992) and Jackson's specifications work (Jackson 1995); the paper positions SDD as the *protocol layer* on which the framework's other elements operate, not as a new discipline. **Graduated delegation** as a calibrat-

able surface descends from SAE J3016's six driving-automation levels (SAE International 2021); the framework's four-dimension calibration is a decomposition of J3016's single "automation level" into independent axes, justified by the absence of the physical constraints that make J3016's collapse defensible in vehicles but not in software. **Agent governance and design** descends from Shavit, Agarwal et al.'s seven operational variables for governing agentic AI (Shavit, Agarwal, et al. 2023) and from Anthropic's *Building Effective Agents* (Anthropic 2024a); the framework's four dimensions are a refactoring of a subset of Shavit & Agarwal's variables into orthogonal levers, and the five archetypes are an opinionated reduction of Anthropic's pattern catalogue to a decision-ready taxonomy. **Failure analysis** descends from Cemri et al.'s MAST (Cemri et al. 2025) (empirical multi-agent failure partition into fourteen symptom-locus categories), Zhang et al.'s hallucination survey (Zhang et al. 2025) (model-output-layer sub-classification of Cat 6), and OWASP LLM Top 10 (OWASP Foundation 2025) (attack-surface partition); the framework's Cat 1–7 partitions by *fix locus* (which artifact must change) rather than by symptom or attack surface, and is explicitly complementary to the prior work — §3.4 develops the multi-axis composition.

The framework also acknowledges the broader systems-and-human-factors tradition: Alexander's pattern-language form (Alexander, Ishikawa, and Silverstein 1977) (the framework borrows the form for the archetype catalogue but does not claim Alexander's empirical standard); Brooks's essential-versus-accidental-complexity distinction (Brooks 1975); Meadows's *Thinking in Systems* (Meadows 2008) and Reason's *Human Error* (Reason 1990) (whose active/latent distinction parallels the Cat 6 / Cat 1 distinction). Inference economics is grounded in Pope et al. (Pope et al. 2022); the long-context attention degradation phenomenon is grounded in Liu et al.'s *Lost in the Middle* (Liu et al. 2023). The long-form paper develops each of these in its own subsection; the workshop-length version condenses them here to acknowledge the lineage without re-deriving.

---

3. The framework

3.1 Intent as a design surface

We define **intent** as the human-authored description of what a delegated system should do, the constraints it must respect, and the conditions that distinguish acceptable from unacceptable outcomes. Intent is a designed artifact, separate from three things it is commonly conflated with: *implementation* (what the executor produces, vs. what it was supposed to produce); *requirements* (what stakeholders ask for, vs. what the system author decides to build after the request is interrogated and reduced); and *policy* (organizational or regulatory cross-cutting constraints, vs. system-specific intent). When code becomes the executor, the gap between intent and implementation shows up as bugs; when an LLM agent becomes the executor, the gap shows up as outputs that are syntactically reasonable, individually defensible, and collectively wrong.

Three conditions make the framing acute in 2026 in a way it was not in 1995. The executor is increasingly an LLM agent that defaults to one-shotting through ambiguity rather than escalating, *despite* having the technical means to escalate. The rate of delegation is faster (an agent loop completes a sequence of decisions in seconds where a human team would have completed it in days). The leverage on intent has gone up because the executor's throughput has gone up: an hour spent on a tightly-bounded spec produces work that an agent loop extends into hundreds of correct outputs.

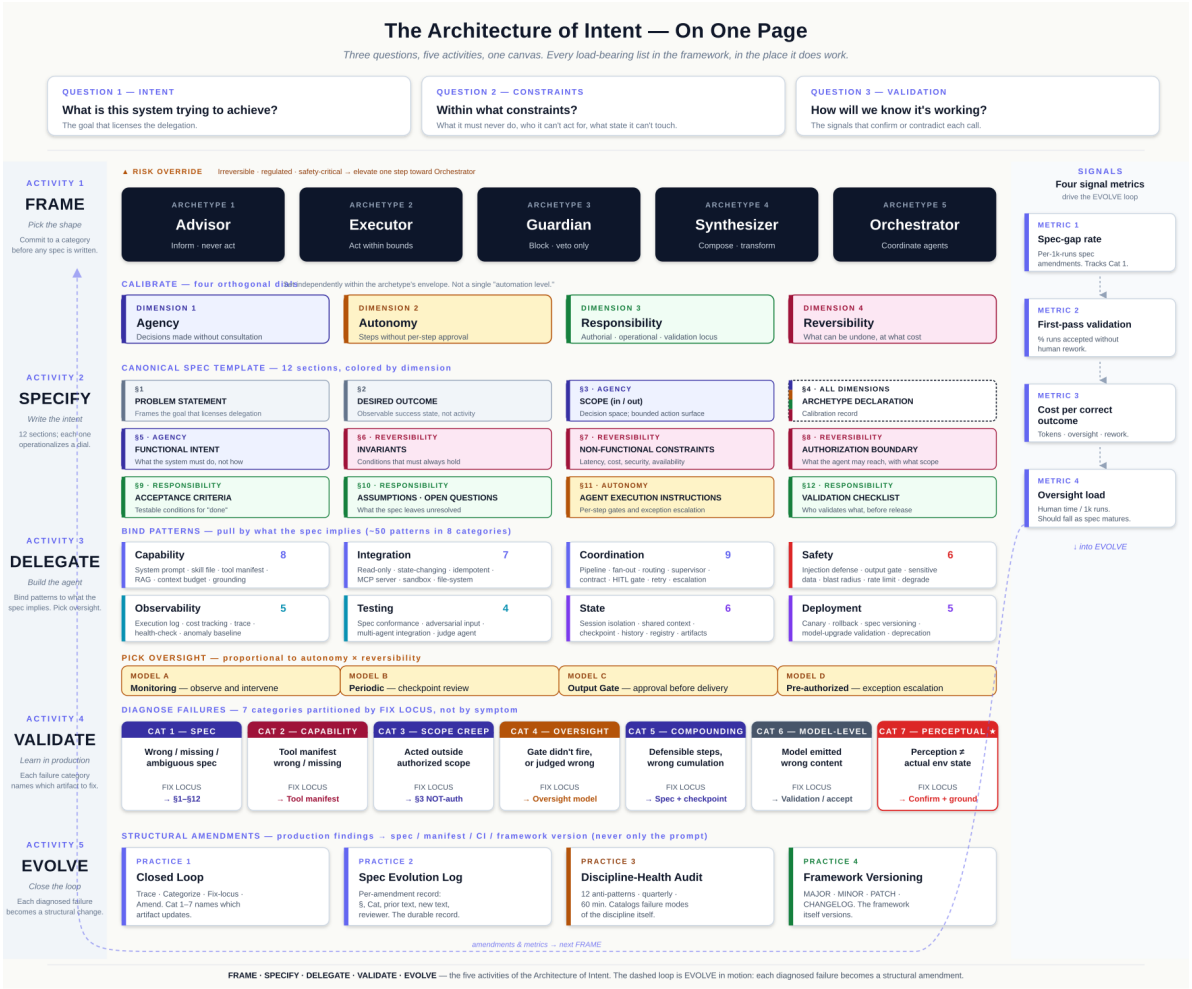

Figure 1: **Figure 1.** The Architecture of Intent on one page. Three questions every delegated system answers (top); five activities that work them out (Frame · Specify · Delegate · Validate · Evolve, on the spine); the load-bearing constructs each activity binds — five archetypes, four orthogonal calibration dimensions, twelve canonical spec sections (colored by the dimension each operationalizes), eight pattern categories, four oversight models, seven fix-locus failure categories, four practices of the closed loop; and four signal metrics on the right rail that descend into the EVOLVE row, where each diagnosed failure becomes a structural amendment that feeds the next intent.

3.2 Archetypes

We propose five canonical archetypes — *Advisor*, *Executor*, *Guardian*, *Synthesizer*, *Orchestrator* — as the smallest decision-ready taxonomy that covers the delegation shapes practitioners deploy.

Archetype	Core function	Agency level	Risk posture	Default oversight
Advisor	Surface information, options, recommendations; never act	Minimal	Low	Human decides and acts
Executor	Carry out well-defined tasks autonomously within bounds	High	Medium	Pre-approved scope; exception-based escalation
Guardian	Enforce rules, validate integrity, block violations	Low (veto)	Low	Alerts; humans resolve
Synthesizer	Aggregate or compose multi-source into a coherent artifact	Moderate	Medium	Output review above threshold
Orchestrator	Coordinate multiple agents or services toward a compound goal	High	High	Active oversight; escalation paths required

A four-question selection tree (Figure 2) resolves archetype assignment in increasing decision difficulty: external behavior, primary function, relationship to other systems, output shape — defaulting to *Executor*. A *risk override* elevates the assigned archetype one step toward Orchestrator when the system's actions touch high-consequence domains (irreversible state changes, regulated data, safety-critical control).

Pre-commitment, not classification. Archetype is committed *before* design, not inferred after. A spec author who writes the spec first and then asks "is this an Advisor or an Executor?" conflates intent with implementation. The pre-commitment makes downstream design decisions traceable: oversight model is determined by archetype; capability boundary is determined by archetype; invariants follow from archetype.

Composition is first-class. A single deployment frequently hosts more than one archetype (Orchestrator over Executors, with a Guardian gating each Executor's output). Three classes of system, increasingly common as of 2026, sit *between* archetypes: coding agents (move between Synthesizer and Executor modes within a session), deep-research agents (Synthesizer with recursive Orchestration), and self-improving systems (honestly two-system deployments). The framework's commitment is that composition is a *first-class design surface* — one governing archetype, embedded components or modes, declared transitions, cross-mode invariants — rather than that the

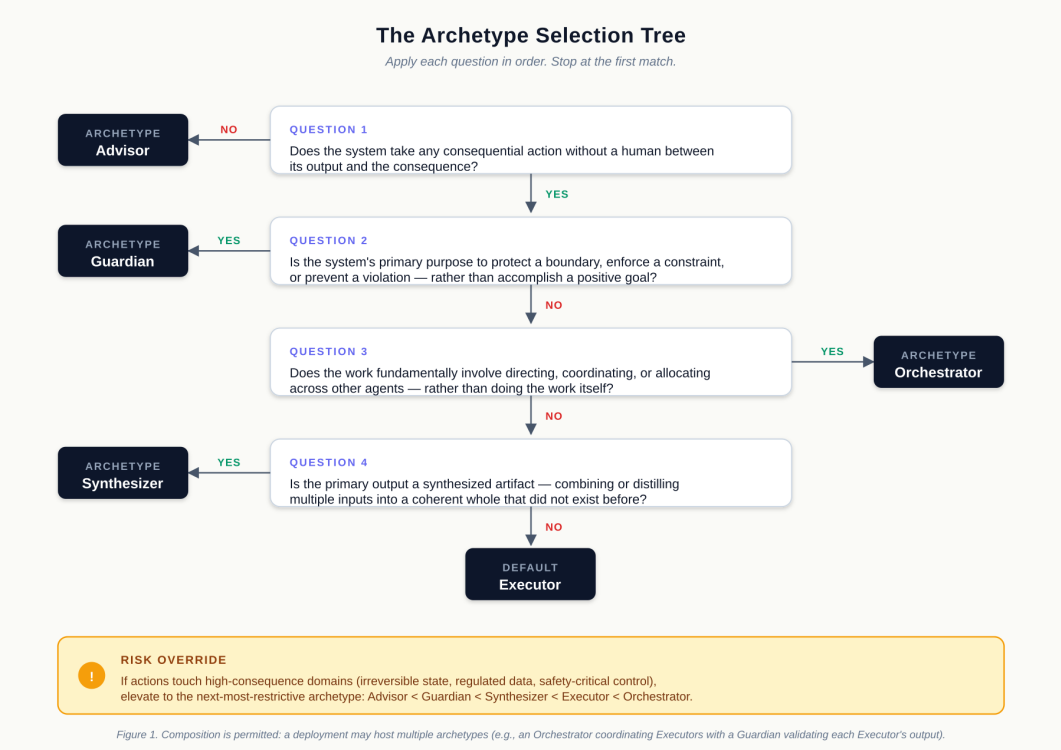

Figure 2: **Figure 2.** The archetype selection tree. Apply questions in order; stop at the first match. The risk-override sidebar elevates the assigned archetype when the system's actions touch high-consequence domains. Composition is permitted: a single deployment may host multiple archetypes.

taxonomy needs a sixth archetype. Adding a sixth would have to name a primary act not on the list; the pressure-point classes do not have a sixth act, they have several of the existing five used in sequence. The companion book (Aldecoa 2026) develops the Composition Declaration sub-block of the canonical spec template.

3.3 Four dimensions of calibration

Within an archetype's envelope, the framework calibrates four orthogonal dimensions:

- **Agency.** The capacity to choose actions in pursuit of a goal — the system's *decision space*. A code-review Advisor exercises narrow agency; an autonomous coding agent exercises wide agency.
- **Autonomy.** Operational independence from per-step human authorization — the system's *execution path*. A deterministic CI/CD pipeline exercises wide autonomy; a compliance-review system that surfaces every flag for human resolution exercises narrow autonomy.
- **Responsibility.** The locus of accountability for outcomes, decomposed into *authorial* (who specified the system), *operational* (who is executing this run), and *validation* (who validated this run's output).
- **Reversibility.** The capacity to undo or recover from an incorrect action. A property of the *system's environment and tooling*, not of the system's intent — and itself a design lever (soft deletes, draft queues, audit logs supporting replay, undo buffers, approval gates are all reversibility-extending mechanisms).

Orthogonality. These four dimensions are independent. Figure 3 plots two of the four (Agency × Autonomy); all four quadrants contain real deployments. A compliance Guardian exercises wide agency per call (judging whether a transaction is suspicious) but narrow autonomy (every call surfaces for human approval). A deterministic CI/CD pipeline exercises no judgment per step (narrow agency) but executes the entire sequence without per-step approval (wide autonomy). The other two pairs are similarly orthogonal.

The orthogonality argument matters because when the four are treated as a single automation level, spec authors have one lever to pull. J3016's collapse (SAE International 2021) is defensible for vehicles (physics constrains the dimensions to move together); software systems have no equivalent constraint. The framework's contribution is the *operationalization of the orthogonality* — Shavit & Agarwal's seven variables already covered the conceptual ground.

Each dimension maps to specific spec clauses: agency to §3 and §4 (authorized and NOT-authorized scope); autonomy to §4's oversight model; responsibility to §1 (owner) and §12 (validation checklist); reversibility to §7 (tool manifest) and §8 (authorization boundary). The framework's insistence is that all four mappings be present with explicit values in every spec.

The framework's working position is that **cost is *not* a fifth calibration dimension** — cost is partly *derived* from the four behavioral dimensions, and is a *resource* commitment rather than a *behavioral* one. The companion book develops a *Cost Posture* sub-block of §4 that captures the resource commitment alongside the Composition Declaration without promoting it to a dimension.

3.4 The fix-locus failure taxonomy (Cat 1–7)

When a delegated system produces a wrong outcome, the operationally useful question is not *what failed* but *which artifact must change so this doesn't happen again*. We partition failures into seven categories by *fix locus* — the artifact whose modification prevents recurrence.

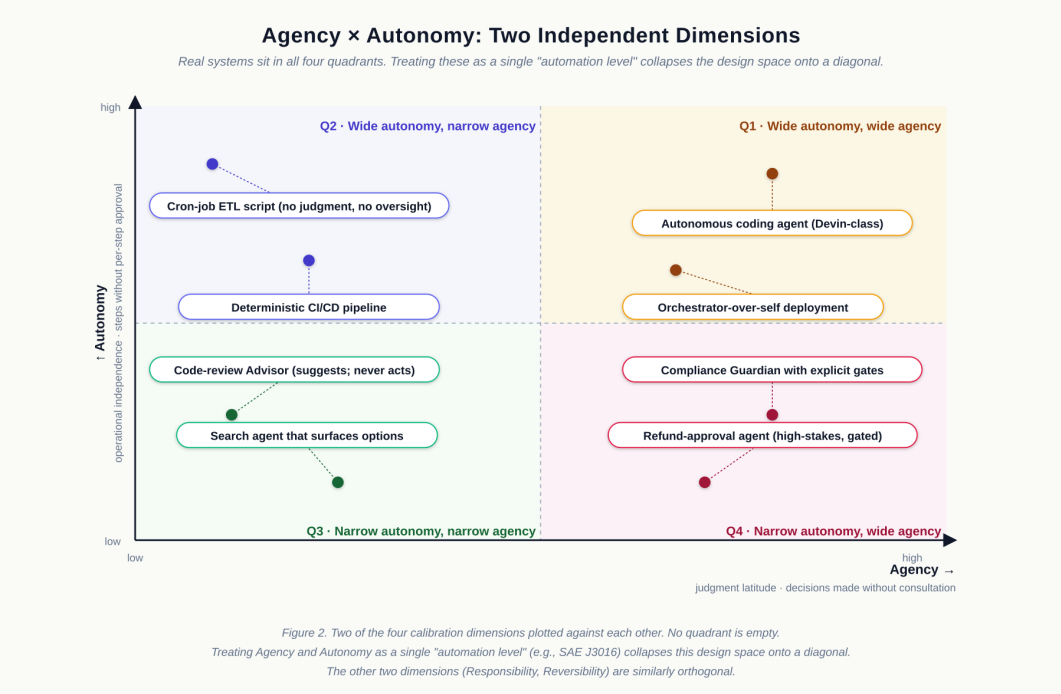

Figure 3: **Figure 3.** Two of the four calibration dimensions plotted against each other. Each quadrant has real deployments; no quadrant is empty. Treating Agency and Autonomy as a single "automation level" — as SAE J3016 does for driving (SAE International 2021) — collapses this design space onto a diagonal.

Category	Failure shape	Fix locus
Cat 1 — Spec	Wrong, missing, or ambiguous spec	The spec
Cat 2 — Capability	Tool manifest wrong / missing / over-scoped	Tool manifest; capability authorization
Cat 3 — Scope creep	Agent acted outside authorized scope	Spec NOT-authorized clauses; agent system prompt
Cat 4 — Oversight	Gate didn't fire, or judged wrong	Oversight model; gate configuration
Cat 5 — Compounding	Defensible steps, wrong cumulation	System spec; checkpoint pattern
Cat 6 — Model-level	Model emitted wrong content despite correct spec/manifest/oversight	Structural validation; allowlist; accept residual risk
Cat 7 — Perceptual	**Perceiving-then-acting system's perception does not match actual environment state**	**Confirmation gate; screenshot-then-verify; multimodal grounding; element-allowlist**

The first six synthesize common practice. The seventh, *Perceptual Failure*, is novel and addresses computer-use agents (Anthropic 2024c; OpenAI 2025; Google 2025) whose failures include shapes that prior taxonomies — MAST (Cemri et al. 2025), Zhang et al.'s hallucination survey (Zhang et al. 2025), OWASP LLM Top 10 (OWASP Foundation 2025) — do not partition. The system's perception of the screen diverges from the screen's actual state, and the system acts on the wrong perception. Cat 7 has four sub-categories (misidentification, missed element, hallucinated element, state miscount) each with a structural fix (confirmation gate, screenshot-then-verify, element-allowlist with DOM grounding, re-verification at moment of use). The sub-categories are not exhaustive — perception failure modes will continue to be discovered as more computer-use deployments surface them — but they cover the shapes practitioners report most often as of 2026.

Fix locus vs. symptom locus. A failure's symptom is *what was observed* (a wrong refund amount); its fix locus is *the artifact whose modification eliminates recurrence* (the spec that authorized the wrong amount). The two diverge frequently. The fix-locus framing is operationally useful because it tells the team which artifact to update, which is what post-incident response requires. The framework's Cat 1–7 is *complementary* to MAST's fourteen-category symptom partition and to the OWASP attack-surface partition: a serious post-incident analysis benefits from all three labels (MAST symptom, OWASP surface if applicable, Cat 1–7 fix locus), pointing at distinct artifacts that may need to change. The framework's contribution at this layer is the fix-locus *axis*, which the prior partitions do not provide.

3.5 Spec-Driven Development as the protocol

Spec-Driven Development (SDD) is the discipline of writing a complete, validated specification *before* the executor runs, treating the spec as the authoritative source against which output is reviewed, and updating the spec rather than the output when execution reveals the spec was wrong. SDD predates this paper (GitHub 2024; Microsoft 2026); what SDD's 2024–2026 form adds is the recognition that agentic execution makes spec-first practice non-optional. The execution-first

bias documented in §1 is the strongest argument for the discipline: if the agent reliably escalated ambiguity, a free-form prompt and a clarification API would suffice. Because it does not, ambiguity resolution must move out of the agent and into the spec lifecycle — concretely, GitHub spec-kit's `/speckit.clarify` command (GitHub 2024) surfaces ambiguities for human resolution before the agent acts.

A SDD specification in the framework's canonical form contains twelve sections (problem statement, desired outcome, scope/in-and-out, archetype declaration, functional intent, invariants, non-functional constraints, authorization boundary, acceptance criteria, assumptions and open questions, agent execution instructions, validation checklist) plus a spec evolution log. The four-dimension calibration of §3.3 maps to specific clauses; the seven failure categories of §3.4 map to specific update sites — Cat 1 updates §3/§4/§6 depending on the gap, Cat 2 updates §7, Cat 4 updates the oversight clauses in §4, Cat 7 updates §11 with verification-before-action requirements. The mapping makes the framework operational: a failure category dictates which clause to update, and the spec evolution log records the change. The companion book develops the canonical template, the Composition Declaration sub-block of §4, the Cost Posture sub-block of §4, the Intent Design Session ritual that produces a calibrated commitment for one specific system, and the Discipline-Health Audit (a quarterly cadence against the twelve anti-patterns that name how the discipline itself decays).

The framework, in summary: *intent as a designed artifact* (§3.1), shaped by *archetypes* (§3.2), calibrated along *four orthogonal dimensions* (§3.3), diagnosed against *seven failure categories* (§3.4), expressed and evolved through *Spec-Driven Development* (§3.5).

---

4. Application to AI agent systems

The framework's most-acute current application is AI agent systems. The agentic development lifecycle exemplified by Microsoft DevSquad Copilot (Microsoft 2026) runs eight phases (*envisioning* → *spec* → *plan* → *decompose* → *implement* → *learn-in-the-open* → *review* → *refine*) with twelve named specialist agents plus a conductor. The framework's elements compose with this lifecycle naturally: archetype declaration in *spec*, calibration in *spec* and *plan*, failure-taxonomy classification in *review* and *refine*. The framework adds the discipline that ensures each phase produces an artifact the next phase can execute against; DevSquad provides the cadence and the agentic tooling.

Capability boundaries via MCP. The Model Context Protocol (Anthropic 2024b) is the protocol layer through which the framework's *least capability* principle becomes operationally enforceable. Each agent receives a tool manifest declaring exactly which tools it can call; MCP servers expose tools with explicit scope. The framework's contribution is the spec-side commitment that the manifest is part of §7 of the spec, not a deployment detail, so the authorization boundary is visible in the artifact humans review.

Coding agents. Cursor, Cline, Devin, and Claude Code (Anthropic 2024--2025) are the most-deployed agent class of 2024–2026. The framework treats them as compositions: Executor with embedded Synthesizer (or, in higher-autonomy deployments, Orchestrator-over-self with Executor and Synthesizer modes). The capability boundary at the file-system, network, and shell layers is operationalized through the tool manifest; the test-protection invariant goes in §6; the explicit decision *against* Devin-style autonomy (when made) goes in the Composition Declaration. The companion book develops a worked pilot end-to-end.

Computer-use agents. Anthropic Computer Use (Anthropic 2024c), OpenAI Operator (OpenAI 2025), and Gemini computer use (Google 2025) perceive a screen via vision and act via simulated input. They are the deployment class for which Cat 7 (Perceptual Failure) was added. The framework's structural controls — confirmation gate before high-consequence actions, screenshot-then-verify, element-allowlist with DOM grounding where DOM is available, re-verification of position-based facts at the moment of use — operationalize the §11 fix-locus for Cat 7.

The long-form paper develops each subsection more thoroughly and includes Appendix B (a phase-by-phase mapping of the framework's elements to DevSquad's eight phases). This workshop variant points readers to that mapping rather than reproducing it; the framework's compositional clarity with DevSquad is the central application claim, not the phase-by-phase mechanics.

---

5. Discussion

When the framework helps. The framework's overhead — archetype commitment, four-dimension calibration, spec authoring against a twelve-section template, failure-taxonomy classification, living-spec evolution — pays back when three conditions hold: *non-trivial autonomy* (the executor makes judgment-laden choices wider than the spec author can enumerate); *consequential outcomes* (failure costs exceed the overhead of precise specification by a comfortable margin); *sustained operation* (the system runs more than a handful of times). When any one fails — a one-off prototype, a low-consequence deployment, a single-use system — teams should adopt the vocabulary (archetypes, dimensions, failure categories) without the full SDD apparatus. The vocabulary is cheap; the apparatus earns its keep on serious deployments.

When it doesn't. Regulated industries (healthcare, finance, defense, aviation) have compliance regimes whose specification requirements are stricter than the framework's canonical template; the framework composes with these regimes but does not substitute. Multi-organizational agent systems (agents from different organizations interacting through standardized protocols) surface governance problems the framework does not solve. The companion book adds (v2.4.0) a chapter on *multi-tenant fleet governance* covering the four structural moves a platform team needs — constraint inheritance hierarchy, cross-tenant isolation contract, fleet-partitioned telemetry, platform-tier failure-locus rule — to scale single-system governance to a fleet of tenant teams sharing infrastructure. The chapter is explicit that those four moves carry a fleet from one to ~50 tenants; beyond that, additional org machinery is needed that this paper does not develop.

Generalization beyond AI agents. The framework's central claim is that intent is a primary design surface for *any* delegated system. AI agent systems are the most-acute current instance because the delegation is wider and faster than at any prior point in software practice, but the four elements — archetype, four-dimension calibration, fix-locus failure taxonomy, spec-as-control-surface — are not specific to LLM-driven executors. Organizational delegation recurs recognizably (Advisor maps to analyst functions, Executor to operations functions, Guardian to compliance, Synthesizer to planning, Orchestrator to program management); CI/CD pipelines are delegated executors of release decisions. Empirical validation of the framework's value in non-agent domains is future work.

Complementarity with prior failure work. A serious post-incident analysis benefits from labels on three axes: MAST's empirical symptom category (Cemri et al. 2025), OWASP's attack surface (OWASP Foundation 2025) where applicable, and Cat 1–7's fix locus. The labels point at

distinct artifacts that may need to change.

---

6. Limitations

- **Position-paper status.** The paper is offered as a structural design tool, not as an empirical intervention. Claims are normative ("you should design this way") rather than descriptive ("we measured what works").
- **No empirical validation at scale.** The companion book provides three worked examples (customer support, code-generation pipeline, internal docs Q&A); the long-form paper walks one pilot end-to-end (§5 of that version) as an existence proof. Quantitative validation across many deployments is future work.
- **Archetype taxonomy is opinionated.** Five is a working choice, not a derived classification.
- **Cat 7 is preliminary.** The category is named but not yet stress-tested across many computer-use deployments. Sub-categories may need revision as more deployments surface failure modes.
- **Generalization beyond AI agents is asserted, not demonstrated.** §5 argues the framework generalizes; we do not provide worked examples for non-agent applications.
- **Fleet governance scales to ~50 tenants.** The companion book's v2.4.0 chapter on multi-tenant fleet governance develops the *first* layer of fleet discipline. Hundreds of tenants require additional infrastructure-organizational machinery that this paper does not undertake.
- **Vocabulary friction.** The paper introduces coined or refactored terms (intent as a design surface, fix-locus taxonomy, Cat 7). Adoption requires linguistic uptake.

---

7. Conclusion

The cost of imprecise intent has always existed in software. It was tolerable for decades because human professionals exercised silent judgment to bridge it. As delegation widens — to LLM-driven agents in 2024–2026, to automated pipelines, to organizational roles — the executor and the judgment layer separate. Modern agent harnesses provide clarification APIs, but the agents using them exhibit a documented execution-first bias and rarely invoke escalation at moments that would have warranted it. The cost stops being absorbed silently and starts surfacing as wrong outputs, scope drift, and post-incident churn.

The Architecture of Intent proposes a structural response. Treat intent as a primary design artifact distinct from implementation. Commit to one of five archetypes before designing the system. Calibrate four orthogonal dimensions — agency, autonomy, responsibility, reversibility — independently rather than collapsing them onto a single automation level. Diagnose failures by *fix locus* across seven categories, with Cat 7 (Perceptual Failure) added for perceiving-then-acting systems. Express the result through Spec-Driven Development as the executable protocol.

The framework's bet is that as delegation widens, intent precision compounds in value. A spec author whose hour produces a tightly-bounded specification commits the executor to hundreds of correct outputs; the same hour spent on a single output commits one. The leverage on intent has always been favorable; what 2024–2026 changes is that the leverage is now collectible. The framework is offered as the structure that makes the collection routine — for AI agent systems specifically, and for delegated systems more broadly. The long-form paper develops each section in

greater depth and walks one pilot end-to-end as an existence proof of the discipline; the companion book is the practitioner's working manual.

References

Bibliography rendered here at compile time from `references.bib`.

Aldecoa, Marcel. 2026. *The Architecture of Intent: A Field Guide to Designing and Shipping AI Agent Systems*. Self-published.

Alexander, Christopher, Sara Ishikawa, and Murray Silverstein. 1977. *A Pattern Language: Towns, Buildings, Construction*. Oxford University Press.

Anthropic. 2024--2025. "Claude Code: an agentic coding tool." https://docs.anthropic.com/claude/docs/claude-code.

———. 2024a. "Building Effective Agents." https://anthropic.com/research/building-effective-agents.

———. 2024b. "Introducing the Model Context Protocol." https://modelcontextprotocol.io.

———. 2024c. "Introducing computer use, a new Claude 3.5 Sonnet, and Claude 3.5 Haiku." https://anthropic.com/news/3-5-models-and-computer-use.

Brooks, Frederick P. 1975. *The Mythical Man-Month: Essays on Software Engineering*. Addison-Wesley.

Cemri, Bora et al. 2025. "MAST: A Multi-Agent System failure Taxonomy." *arXiv Preprint*.

GitHub. 2024. "spec-kit: Toolkit for spec-driven development with AI assistants." https://github.com/github/spec-kit.

Google. 2025. "Gemini computer use." https://ai.google.dev.

Jackson, Michael. 1995. *Software Requirements & Specifications: A Lexicon of Practice, Principles and Prejudices*. Addison-Wesley.

Liu, Nelson F., Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni, and Percy Liang. 2023. "Lost in the Middle: How Language Models Use Long Contexts." *arXiv Preprint arXiv:2307.03172*.

Meadows, Donella H. 2008. *Thinking in Systems: A Primer*. Chelsea Green Publishing.

Meyer, Bertrand. 1992. "Applying 'Design by Contract'." *Computer* 25 (10): 40–51.

Microsoft. 2026. "DevSquad Copilot: An eight-phase agentic development lifecycle." https://github.com/microsoft/devsquad-copilot.

OpenAI. 2025. "Introducing Operator." https://openai.com/index/introducing-operator.

OWASP Foundation. 2025. "OWASP LLM Top 10 (2025 update)." https://genai.owasp.org/llm-top-10.

Pope, Reiner, Sholto Douglas, Aakanksha Chowdhery, Jacob Devlin, James Bradbury, Anselm Levskaya, Jonathan Heek, Kefan Xiao, Shivani Agrawal, and Jeff Dean. 2022. "Efficiently Scaling Transformer Inference." *arXiv Preprint arXiv:2211.05102*.

Reason, James. 1990. *Human Error*. Cambridge University Press.

SAE International. 2021. "J3016: Taxonomy and Definitions for Terms Related to Driving Automation Systems for On-Road Motor Vehicles." SAE International.

Shavit, Yonadav, Sandhini Agarwal, et al. 2023. "Practices for Governing Agentic AI Systems." OpenAI. https://openai.com/research/practices-for-governing-agentic-ai-systems.

Zhang, Yifan et al. 2025. "LLM-based Agents Suffer from Hallucinations: A Survey of Taxonomy, Methods, and Directions." *arXiv Preprint arXiv:2509.18970*.

Appendix: Reading paths

This workshop-length variant compresses the long-form arXiv version. For each topic, the table below names the section of this paper and the chapter of the companion book where the topic is developed in full.

Topic	This paper (workshop)	Long-form paper	Companion book
The judgment gap	§1	§1.1–§1.2	Prologue; Introduction
Intent as a design surface	§3.1	§3.1	foundations/02-intent-vs-implementation
Five archetypes (incl. composition first-class)	§3.2	§3.2	frame/02–06; repertoires/02
Four orthogonal dimensions	§3.3	§3.3	foundations/03; frame/03
Cat 1–7 fix-locus failure taxonomy	§3.4	§3.4	foundations/05; delegate/09 for Cat 7
Spec-Driven Development; canonical template	§3.5	§3.5	Part 2 (Specify); especially specify/07
Coding agents	§4	§4	delegate/08; examples/03-coding-agent
Computer-use agents	§4	§4	delegate/09
DevSquad Copilot composition	§4	§4 + Appendix B	operate/06-devsquad-mapping; operate/07-co-adoption-with-devsquad
Multi-tenant fleet governance	§5 (one paragraph)	§5–§6 (mentioned)	operate/08-multi-tenant-fleet-governance
Generalization beyond AI agents	§5	§6.4	Multiple chapters; framework asserts generalization, book elaborates per scenario
Limitations	§6	§7	Introduction §“Honest scope”; evolve/15 anti-patterns
Working practices not in this paper (Intent Design Session, Discipline-Health Audit)	(omitted)	(omitted)	foundations/07; evolve/15

The workshop variant is the gateway; the long form is the citation reference; the book is the working manual.